

Jackett Integration

Revision: 1 Last modified: 2026-06-06T00:00:00Z

Jackett is a proxy server that translates queries from apps like qBittorrent into tracker-site-specific HTTP queries, parses the HTML or JSON responses, and returns results in a unified format. It supports 500+ public and private trackers and handles Cloudflare challenges, site redesigns, and CAPTCHAs automatically.

What This Integration Does

When enabled, Jackett acts as a **meta-tracker** in the merge service fan-out:

- A single jackett tracker entry queries **all configured Jackett indexers** simultaneously
- Results are deduplicated and merged with other tracker sources
- Dead public trackers (Cloudflare-blocked or permanently broken) are automatically backfilled by Jackettâ€™s working indexers

Architecture

[illegible][illegible]

Auto-Discovery (Fully Automatic)

No manual configuration is required. The stack handles everything:

1. `start.sh` **starts Jackett first** via `docker-compose up` (Jackett has `depends_on` priority)
2. **Jackett generates its API key** on first start and stores it in `config/jackett/Jackett/ServerConfig.json`
3. `start-proxy.sh` **reads the key** from the mounted config file (`/jackett-config/Jackett/ServerConfig.json`) and exports `JACKETT_API_KEY`
4. **The merge service sees the env var** and automatically adds `jackett` to the enabled tracker list
5. `start.sh` **also updates** `.env` with the discovered key for host-side convenience

Startup Flow

```
$ ./start.sh
[INFO] Starting qBitTorrent container...
```

```
[INFO] Waiting for Jackett to be ready...
[SUCCESS] Jackett is ready
[SUCCESS] Jackett API key auto-configured in .env
[INFO] Waiting for container to be ready...
...
[SUCCESS] qBittorrent Web UI: http://localhost:7185
[SUCCESS] Dashboard: http://localhost:7187/
[SUCCESS] Jackett Admin: http://localhost:9117/UI/Dashboard
```

Manual Override

If you prefer to set the key manually (e.g., using an existing Jackett instance):

```
# .env
JACKETT_API_KEY=your-real-api-key-here
JACKETT_URL=http://localhost:9117 # optional, defaults to localhost:9117
```

The manual value takes precedence over auto-discovery because `start-proxy.sh` only auto-discovers when the env var is empty.

Container Configuration

docker-compose.yml

```
jackett:
  image: lscr.io/linuxserver/jackett:latest
  container_name: jackett
  environment:
    - PUID=1000
    - PGID=1000
    - TZ=Europe/Moscow
    - AUTO_UPDATE=true
  volumes:
    - ./config/jackett:/config
  network_mode: host
  restart: unless-stopped
  healthcheck:
    test: ["CMD-SHELL", "curl -sf http://localhost:9117/health || exit 1"]
    interval: 30s
    timeout: 10s
    retries: 5
    start_period: 60s
```

Proxy Container Mount

Both proxy containers mount the Jackett config as read-only:

```
volumes:
  - ./config/jackett:/jackett-config:ro
```

This allows `start-proxy.sh` to read the API key without restarting containers.

Plugin Configuration

plugins/jackett.json

```
{
  "api_key": "YOUR_API_KEY_HERE",
  "thread_count": 20,
  "tracker_first": false,
  "url": "http://localhost:9117"
}
```

This file is installed to `config/qBittorrent/nova3/engines/jackett.json` by `install-plugin.sh` and `setup.sh`. The `api_key` field is automatically overridden by the `JACKETT_API_KEY` environment variable when present.

Environment Variable Precedence

The patched `jackett.py` plugin checks env vars **after** loading the JSON file:

1. Load `jackett.json`
2. Override `api_key` with `JACKETT_API_KEY` env var (if set)
3. Override `url` with `JACKETT_URL` env var (if set)

This means: - You can leave `jackett.json` untouched - Container orchestration injects the real key via env vars - Local development can override via `.env`

Merge Service Integration

Tracker Registration

`download-proxy/src/merge_service/search.py` adds Jackett to `_get_enabled_trackers()`:

```
if os.getenv("JACKETT_API_KEY") and os.getenv("JACKETT_API_KEY") != "YOUR_API_KEY_HERE":
    trackers.append(TrackerSource(name="jackett", url="http://localhost:9117"))
```

Search Dispatch

Jackett is routed through the same NDJSON subprocess pipeline as public trackers:

```
elif tracker.name in PUBLIC_TRACKERS or tracker.name == "jackett":
    results = await self._search_public_tracker(tracker.name, query, category)
```

This ensures: - Deadline-based timeout (configurable via `PUBLIC_TRACKER_DEADLINE_SECONDS`) - Streaming NDJSON result capture - Per-plugin diagnostic collection

Adding Indexers to Jackett

1. Open `http://localhost:9117/UI/Dashboard`
2. Click **Add indexer**

3. Search for your desired tracker (e.g., `1337x`, `RARBG`, `EZTV`)
4. Click the + button to add
5. Configure credentials if required (private trackers)

All configured indexers are automatically included in merge service searches – no restart required.

Troubleshooting

Jackett not appearing in search results

```
# Check if the API key was auto-discovered
curl -s http://localhost:7187/health | jq .

# Check proxy container logs
podman logs qbittorrent-proxy | grep -i jackett

# Verify the config file exists
ls -la config/jackett/Jackett/ServerConfig.json
cat config/jackett/Jackett/ServerConfig.json | jq '.APIKey'

# Check env var inside the container
podman exec qbittorrent-proxy sh -c 'echo $JACKETT_API_KEY'
```

Plugin shows `api key error`

This means `JACKETT_API_KEY` is not set or is still the placeholder. Check:

1. Jackett container is healthy: `curl -sf http://localhost:9117/health`
2. Config file was generated: `cat config/jackett/Jackett/ServerConfig.json`
3. Proxy container has the mount: `podman exec qbittorrent-proxy ls /jackett-config/`

Slow Jackett searches

Jackett queries **all enabled indexers** in parallel. If you have many indexers configured:

- Increase `PUBLIC_TRACKER_DEADLINE_SECONDS` in `.env` (default: 15, max: 120)
- Remove unused indexers from the Jackett dashboard
- Reduce `thread_count` in `plugins/jackett.json` if Jackett is CPU-bound

Test Jackett independently

```
# Get your API key
API_KEY=$(cat config/jackett/Jackett/ServerConfig.json | jq -r '.APIKey')

# Test a search
curl "http://localhost:9117/api/v2.0/indexers/all/results/torznab/api?apikey=$API_KEY"
```

Security Notes

- The Jackett config mount is **read-only** (:ro) in proxy containers
- The API key is never logged; `CredentialScrubber` in `log_filter.py` masks it
- Jackett's WebUI is accessible on localhost only (host networking)
- If you expose Jackett externally, set a strong admin password in the Jackett dashboard

Files Modified for This Integration

File	Change
<code>docker-compose.yml</code>	Added jackett service, mounts, depends_on, env vars
<code>.env</code>	Added JACKETT_API_KEY placeholder
<code>start.sh</code>	Jackett-first startup, auto-key extraction, .env update
<code>start-proxy.sh</code>	Auto-discovers and exports JACKETT_API_KEY
<code>plugins/community/jackett.py</code>	Env-var override for api_key and url
<code>plugins/jackett.json</code>	URL updated to localhost:9117
<code>install-plugin.sh</code>	Copies .json config files alongside plugins
<code>setup.sh</code>	Copies .json config files during setup
<code>download-proxy/src/merge_service/search.py</code>	Jackett tracker registration + dispatch
<code>scripts/extract-jackett-key.py</code>	Standalone key extraction utility

Auto-Configuration (canonical: boba-jackett:7189 + DB)

Jackett indexer auto-configuration is now owned by the dedicated **boba-jackett** Go service (port 7189) backed by a SQLite system database (`config/boba.db`) encrypted with `BOBA_MASTER_KEY`.

The Python `download-proxy` previously exposed a transient `/api/v1/jackett/autoconfig/last` endpoint and an `in-memory lifespan()` hook. Both were removed at commit <see plan 2026-04-27 Â§38>. The canonical surface is now:

- `GET /api/v1/jackett/autoconfig/runs` " paginated history of every auto-config run (persisted, not just last-run-in-memory).
- `GET /api/v1/jackett/autoconfig/runs/{id}` " single run detail.
- `POST /api/v1/jackett/autoconfig/trigger` " manual re-run from the UI.

See `qBittorrent-go/internal/jackett/` and `cmd/boba-jackett/`.

Source-of-truth data layer (DB)

Credentials, overrides, and run history live in `config/boba.db` (modernnc.org/sqlite " pure Go, no CGO). The schema (full reference at `docs/BOBA_DATABASE.md`):

Table	Purpose
<code>tracker_credentials</code>	

Table	Purpose
	per-tracker username / password / cookie blob, encrypted with BOBA_MASTER_KEY (AES-256-GCM)
<code>indexer_map_overrides</code>	explicit <code>tracker_name</code> + <code>indexer_id</code> user overrides (highest match priority)
<code>autoconfig_runs</code>	every auto-config run summary (<code>ran_at</code> , <code>discovered</code> , <code>configured_now</code> , <code>already_present</code> , <code>errors</code>)
<code>jackett_indexer_catalog</code>	last refreshed catalog snapshot from Jackett (browse + filter source for the UI)

File mode is enforced to 0600 on first open by `internal/db.Open`.

.env import is bootstrap-only

On first start (DB empty) or via the `boba-jackett bootstrap` flow, `<NAME>_USERNAME / <NAME>_PASSWORD / <NAME>_COOKIES` triples in `./ .env` are imported into `tracker_credentials` exactly once, then encrypted with the master key. Subsequent edits to `.env` do **not** override DB state – the UI is the canonical edit surface.

The denylist (default `QBITTORRENT`, `JACKETT`, `WEBUI`, `PROXY`, `MERGE`, `BRIDGE`, override `JACKETT_AUTOCONFIG_EXCLUDE`) is still honored at scan time so project-internal env vars never enter the DB.

Matching and configuration

For each tracker credential bundle in `tracker_credentials`:

1. Checks `indexer_map_overrides` for an explicit `tracker_name` + `indexer_id`.
2. Falls back to fuzzy matching (`Levenshtein ratio() >= 0.85`, case-insensitive) against the cached `jackett_indexer_catalog`.
3. Skips ambiguous matches with `skipped_ambiguous`.

For each match, the service fetches the per-indexer config template from Jackett, maps DB-decrypted credentials onto compatible fields (`username/password/cookie/cookies/cookieheader`), and POSTs. Idempotent – already-configured indexers land in `already_present`.

UI

The Angular dashboard at `http://localhost:7187/jackett` is the canonical edit surface. Pages (some shipping in successor patches):

- **Credentials** (shipped): list / add / edit / delete tracker credentials. Edits are encrypted at rest via BOBA_MASTER_KEY.
- **Indexers** (shipped backend, frontend deferred): browse the catalog, force-refresh, configure, test, toggle, remove.
- **Runs** (shipped backend): paginated auto-config run history.

Failure modes (best-effort)

Auto-config is best-effort and **never blocks boot**. Errors land in the runâ€™s errors list and a WARNING log. Codes:

Code	Meaning
jackett_unreachable	Jackett container not accepting connections
jackett_auth_failed	Bad / missing API key (HTTP 401)
jackett_auth_missing_key	JACKETT_API_KEY env var was empty
catalog_parse_failed	Catalog endpoint returned non-JSON
jackett_total_timeout_60s	Hit the outer 60-second cap
indexer_config_failed:<id>:<reason>	Per-indexer failure

Configuration

Env var	Purpose	Default
BOBA_MASTER_KEY	32-byte hex encryption key for tracker_credentials	auto-generated by start.sh and bootstrap.EnsureMasterKey
BOBA_DB_PATH	SQLite path	/config/boba.db
BOBA_ENV_PATH	host .env path used for one-shot bootstrap import	/host-env/.env
JACKETT_INDEXER_MAP	CSV NAME:indexer_id overrides (legacy; prefer DB indexer_map_overrides)	empty
JACKETT_AUTOCONFIG_EXCLUDE	CSV prefix denylist	QBITTORRENT, JACKETT, WEBUI

Files

- qBitTorrent-go/cmd/boba-jackett/main.go â€™ service entry point
- qBitTorrent-go/internal/jackett/ â€™ autoconfig orchestrator
- qBitTorrent-go/internal/db/ â€™ SQLite layer + migrations
- qBitTorrent-go/internal/envfile/ â€™ atomic .env writer
- qBitTorrent-go/internal/bootstrap/ â€™ EnsureMasterKey + .env import
- frontend/src/app/jackett/ â€™ Angular pages
- docs/BOBA_DATABASE.md â€™ schema reference, key lifecycle, backup, recovery